

EJERCICIOS DE REPASO

PROGRAMACIÓN ORIENTADA A OBJETOS

1. Escribe una clase Cuenta para representar una cuenta bancaria. Los datos de la cuenta son: nombre del cliente (String), número de cuenta (String), tipo de interés (double) y saldo (double).

La clase contendrá los siguientes métodos:

- Constructor por defecto
- Constructor con todos los parámetros
- Métodos setters/getters para asignar y obtener los datos de la cuenta.
- Métodos ingreso y reintegro. Un ingreso consiste en aumentar el saldo en la cantidad que se indique. Esa cantidad no puede ser negativa. Un reintegro consiste en disminuir el saldo en una cantidad pero antes se debe comprobar que hay saldo suficiente. La cantidad no puede ser negativa. Los métodos ingreso y reintegro devuelven true si la operación se ha podido realizar o false en caso contrario.
- Método transferencia que permita pasar dinero de una cuenta a otra siempre que en la cuenta de origen haya dinero suficiente para poder hacerla. Ejemplo de uso del método transferencia:
 - `cuentaOrigen.transferencia(cuentaDestino, importe);` que indica que queremos hacer una transferencia desde `cuentaOrigen` a `cuentaDestino` del importe indicado.

Prueba el funcionamiento de la clase Cuenta.

2. Escribe una clase para representar fracciones. La clase tendrá dos atributos de tipo int: num (numerador) y den (denominador). La clase debe contener los constructores y métodos adecuados para que este método main funcione de forma correcta:

```
public static void main(String[] args) {  
    // Se crean 4 fracciones  
    Fraccion f1 = new Fraccion(1, 4); // Fracción 1/4  
    Fraccion f2 = new Fraccion(1, 2); // Fracción 1/2  
    Fraccion f3 = new Fraccion(); // Fracción 0/1  
    Fraccion f4 = new Fraccion(4); // Fracción 4/1  
    // operaciones aritméticas con esas fracciones  
    Fraccion suma = f1.sumar(f2);  
    Fraccion resta = f1.restar(f3);  
    Fraccion producto = f1.multiplicar(f4);  
    Fraccion cociente = f1.dividir(f2);  
    //mostrar resultados  
    System.out.println(f1 + " + " + f2 + " = " + suma);  
    System.out.println(f1 + " - " + f3 + " = " + resta);  
    System.out.println(f1 + " * " + f4 + " = " + producto);  
    System.out.println(f1 + " / " + f2 + " = " + cociente);  
}
```

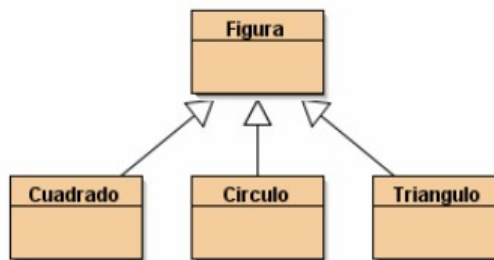
3. Crea una clase llamada Libro que guarde la información de cada uno de los libros de una biblioteca. La clase debe guardar el título del libro, autor, número de ejemplares del libro y número de ejemplares prestados. La clase contendrá los siguientes métodos:

- Constructor por defecto.
- Constructor con parámetros.
- Métodos Setters/getters
- Método préstamo que incremente el atributo correspondiente cada vez que se realice un préstamo del libro. No se podrán prestar libros de los que no queden ejemplares disponibles para prestar. Devuelve true si se ha podido realizar la operación y false en caso contrario.

- Método devolución que decremente el atributo correspondiente cuando se produzca la devolución de un libro. No se podrán devolver libros que no se hayan prestado. Devuelve true si se ha podido realizar la operación y false en caso contrario.
- Método toString para mostrar los datos de los libros. Este método se hereda de Object y lo debemos modificar (override) para adaptarlo a la clase Libro.

Escribe un programa para probar el funcionamiento de la clase Libro.

4. Escribe un programa que implemente la siguiente jerarquía de clases:



Implementar aquellos atributos y métodos necesarios para que se pueda ejecutar el siguiente programa:

```

public class Jerarquía {
    public static void main(String[] args) {
        Vector<Figura> figuras = new Vector<Figura>();
        figuras.add(new Circulo(10)); // Radio=10
        figuras.add(new Cuadrado(10)); // Lado=10
        figuras.add(new Triangulo(10,5)); // Base=10, Altura=5;
        for (Figura f: figuras)
            System.out.println("Área: "+f.area());
    }
}
  
```

Al ejecutar el programa, deberá aparecer por pantalla el área calculada de cada una de las figuras creadas. La clase “Vector” se comporta como un array pero de forma dinámica, puedes sustituirlo por un `ArrayList<Figura>`. Haz que “Figura” sea una clase abstracta.